PageRank and GraphRAG Report

1 Part 1: Analytical Derivation of PageRank

Question :

6) Consider a web graph with Google matrix M where a random web surfer jumps off (i.e., "teleport") to another website (with all webpages equally likely) from the current website with probability p. Derive the PageRank in analytical form (i.e., in closed form) in terms of M and p. Describe how the PageRank may change with respect to p. Explain using illustrative examples.

Figure 1: Question for Part 1

Answer :

Assume a web graph with n webpages. Let M denote the column-stochastic link matrix of the graph, where each column represents the probability of following a hyperlink from one page to another. Dangling pages are handled by replacing their columns with the uniform distribution.

A random web surfer behaves as follows:

- With probability $1 - p$, the surfer follows a hyperlink.
- With probability p, the surfer teleports uniformly to any webpage.

Under this model, the PageRank vector $r \in \mathbb{R}^n$ is the stationary distribution of the surfer.

1.1 PageRank Equation

The PageRank vector satisfies

$$r = (1 - p)Mr + \frac{p}{n}\mathbf{1},$$

where $\mathbf{1}$ is the all-ones vector.

This equation shows that PageRank comes from two sources:

1. the link-following contribution $(1 - p)Mr$,
2. the teleportation contribution $\frac{p}{n}\mathbf{1}$.

Hence, r is the long-run probability distribution of the random surfer.

1.2 Closed-Form Derivation

Rearranging gives

$$r - (1-p)Mr = \frac{p}{n}\mathbf{1}.$$

Therefore,

$$(I - (1-p)M)r = \frac{p}{n}\mathbf{1}.$$

Assuming $p > 0$, the matrix $I - (1-p)M$ is invertible. Thus the closed-form PageRank is

$$\boxed{r = \frac{p}{n}(I - (1-p)M)^{-1}\mathbf{1}}$$

1.3 Why the Inverse Exists

Since M is stochastic, its spectral radius satisfies $\rho(M) = 1$. Because $0 < p < 1$,

$$\rho((1-p)M) = 1 - p < 1.$$

Hence $I - (1-p)M$ is nonsingular, so the inverse exists and the PageRank vector is well-defined and unique for every $p > 0$.

1.4 Effect of p

The parameter p controls the balance between hyperlink structure and random teleportation.

If p is small, the surfer follows links most of the time. In this case, PageRank depends strongly on the graph structure. Highly connected nodes tend to receive much larger rank, and probability mass may accumulate around hubs, sinks, or strongly connected regions.

If p is large, teleportation becomes more important. Then the influence of the hyperlink structure weakens, and the PageRank vector becomes closer to the uniform distribution. In the limiting case,

$$p \to 1 \quad \Rightarrow \quad r \to \frac{1}{n}\mathbf{1}.$$

So increasing p makes the distribution flatter, while decreasing p makes the ranking more concentrated on structurally important pages.

1.5 Illustrative Examples

Star graph.

Consider a graph where many pages point to one central page. When p is small, the central page receives most of the incoming probability mass and has a much higher PageRank than the others. As p increases, teleportation spreads probability more evenly, so the leaf pages gain rank and the center becomes less dominant.

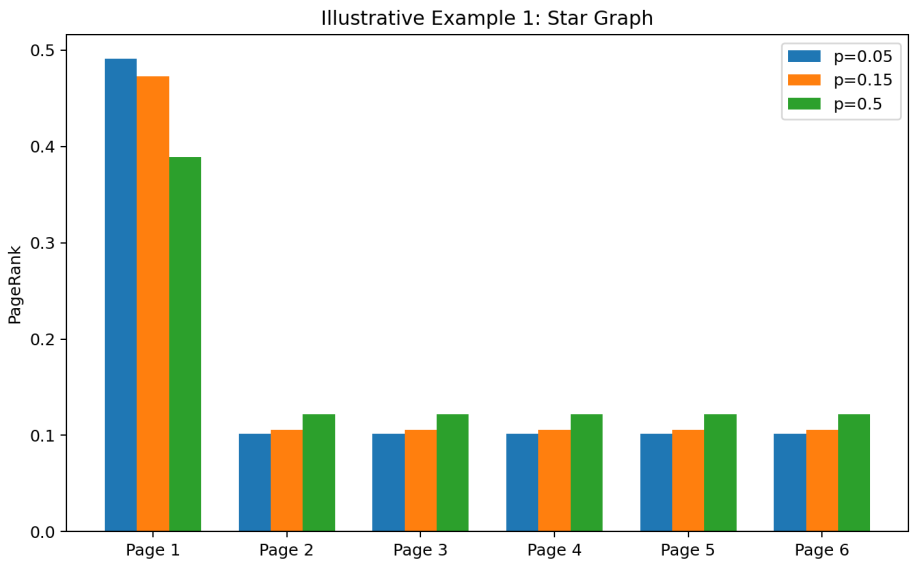

Figure 2: Illustrative star graph example under different values of p.

Chain graph.
Consider a directed chain

$$1 \to 2 \to 3 \to \cdots \to n.$$

When p is small, probability flows forward along the chain and later nodes tend to have larger PageRank. As p increases, teleportation redistributes probability back across all nodes, making the ranking more balanced.

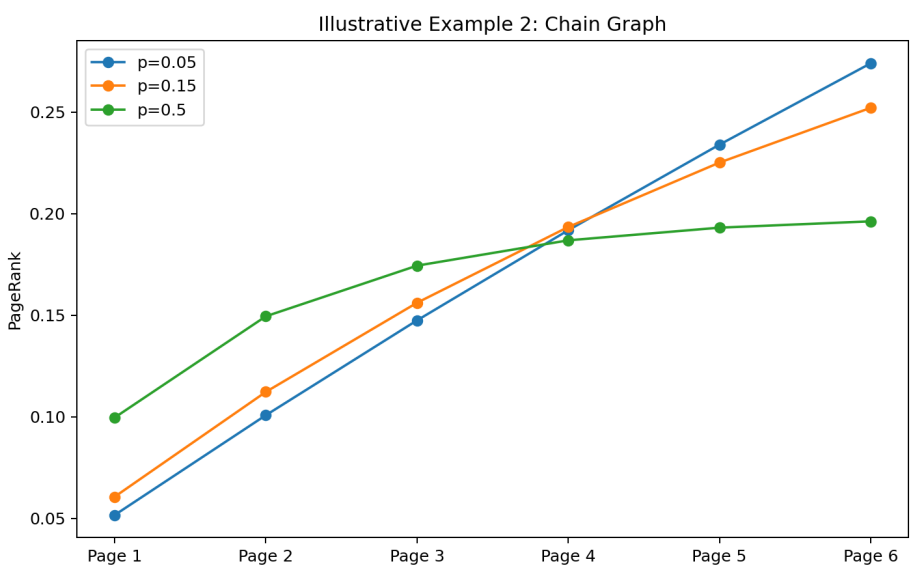

Figure 3: Illustrative chain graph example under different values of p.

Isolated or weakly connected node.

Without teleportation, such a node may receive very little or even zero importance. When $p > 0$, teleportation guarantees that every node receives some probability mass. This prevents pages from being completely ignored and helps ensure a unique ranking.

1.6 Conclusion

The PageRank vector for a web graph with teleportation probability p is

$$r = \frac{p}{n}(I - (1-p)M)^{-1}\mathbf{1}.$$

The parameter p ensures a unique solution and controls how strongly PageRank depends on the hyperlink structure.

2 Part 2: PageRank Implementation and Numerical Evaluation

Question:

Implement the Pagerank algorithm using your favorite programming language (e.g., Python, C++, see link below) and evaluated on large datasets, e.g., web-Google_10k.txt or web-Google.txt provided by Google in 2002 available at:

- 2 -

Cloud Computing

https://hunglvosu.github.io/posts/2020/07/PA3/. You can use any graph processing libraries (see references) or develop your own in your program. Compare the numerical output of your algorithm with the above closed form.

Figure 4: Question for Part 2

Answer:

The PageRank algorithm was implemented in Python and evaluated on the dataset `web-Google_10k.txt`. The implementation uses sparse matrix operations and power iteration to compute the numerical PageRank vector. It also computes the closed-form PageRank by solving the linear system

$$r = \frac{p}{n}(I - (1-p)M)^{-1}\mathbf{1},$$

so that the numerical result can be compared directly with the analytical expression derived in Part 1.

For the experiments, the teleportation probability was set to $p = 0.15$.

2.1 Implementation Summary

The algorithm was implemented as follows:

1. Read the edge list from `web-Google_10k.txt`.
2. Construct the sparse adjacency matrix of the graph.
3. Compute out-degrees and handle dangling nodes by distributing their probability uniformly.
4. Run power iteration until convergence.

5. Compute the closed-form PageRank using a sparse linear solver.
6. Compare the two results using error norms.

2.2 Dataset

The sample dataset contains 10,000 nodes and 78,323 directed edges.

2.3 Numerical Results

Power iteration converged in 142 iterations with final L_1 update

$$9.43 \times 10^{-13}.$$

The difference between the numerical result and the closed-form solution was very small:

$$\|r_{\text{power}} - r_{\text{closed}}\|_1 = 1.86 \times 10^{-12},$$

$$\|r_{\text{power}} - r_{\text{closed}}\|_\infty = 8.01 \times 10^{-14}.$$

These values show that the iterative implementation is consistent with the analytical closed form.

2.4 Top 10 Ranked Pages for $p = 0.15$

Rank	Node ID	PageRank Score
1	486980	0.0069990
2	285814	0.0047475
3	226374	0.0033956
4	163075	0.0033308
5	555924	0.0026861
6	32163	0.0023828
7	828963	0.0021901
8	504140	0.0021481
9	396321	0.0021144
10	599130	0.0021040

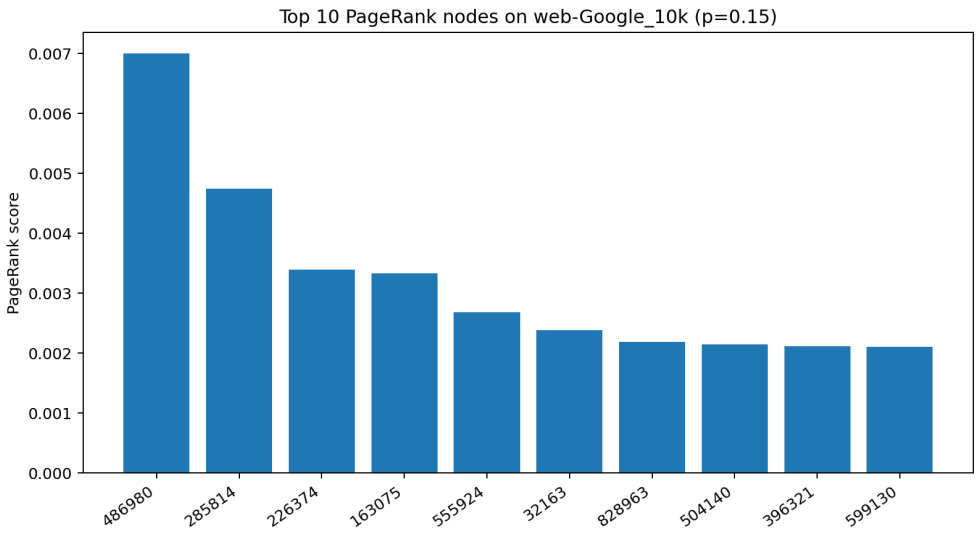

Figure 5: Top 10 PageRank scores on the `web-Google_10k` dataset.

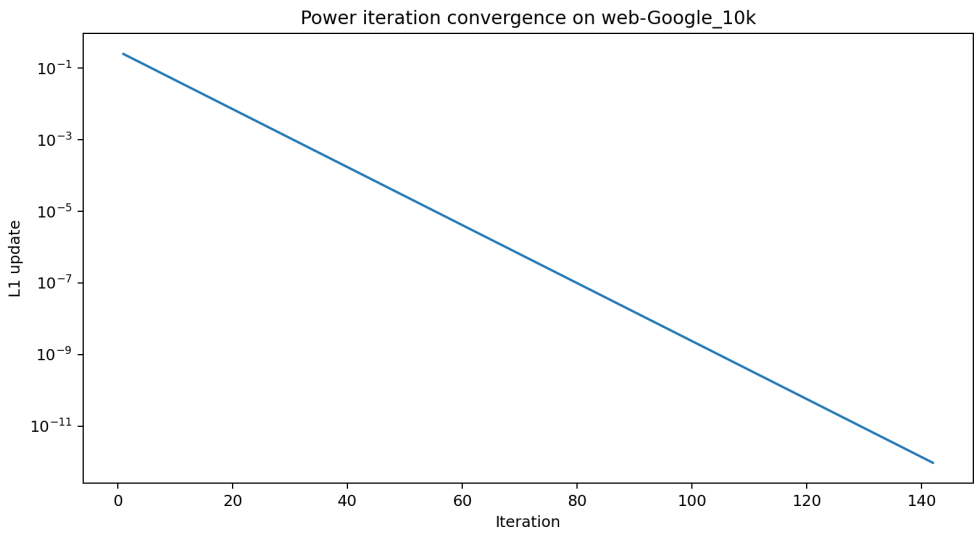

Figure 6: Convergence behavior of power iteration on `web-Google_10k`.

2.5 Discussion

The numerical and closed-form results are almost identical, with errors only at machine precision level. This confirms that the implementation is correct.

Using the power method is also much more practical than evaluating the inverse directly for large graphs. Although the closed-form expression is useful theoretically, directly forming the inverse is computationally expensive. In contrast, power iteration only requires repeated sparse matrix-vector multiplication and therefore scales much better to large web graphs.

2.6 Conclusion

The PageRank algorithm was successfully implemented and tested on `web-Google_10k.txt`. The numerical result obtained by power iteration matches the analytical closed form very closely, which validates both the derivation and the implementation.

3 Part 3: GraphRAG Retrieval with Personalized PageRank

Question:

(ii) Explore the open-source repository fast-graphrag by circlemind-ai on (https://github.com/circlemind-ai/fast-graphrag), which is a GraphRAG framework for interpretable agent-driven retrieval workflows. Its primary goal is to support for dynamic, incrementally updatable knowledge graphs by PageRank to enhance accuracy and dependability. Use GraphRAG to answer a multi-hop query, for instance: "What discoveries by Marie Curie led to later advances in medical imaging?" Injecting Pagerank scores into the retrieval step helps by starting from seed entities ("Marie Curie," "medical imaging") and propagating importance through the graph's connections (e.g., "Marie Curie" -> "discovered radium" -> "used in radiotherapy" -> "advances in medical imaging"), effectively ranking nodes based on their relevance to the specific query context. Write a program/software to take a knowledge graph and a set of query entities to return the top k most relevant nodes.

Figure 7: Question for Part 3

Answer:

According to the open-source repository `fast-graphrag`, which describes itself as a GraphRAG framework for interpretable, high-precision, agent-driven retrieval workflows and states that it currently uses personalized PageRank to explore the graph and find the most relevant pieces of information for a query. [1]

3.1 System Idea

The key idea is to start from a small set of query entities and then propagate relevance through graph connections instead of relying only on direct keyword matching. In this way, multi-hop evidence can be surfaced naturally.

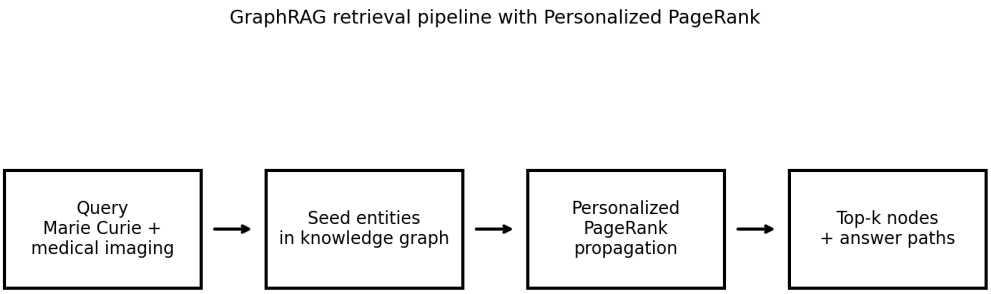

Figure 8: GraphRAG retrieval pipeline using query seeds and personalized PageRank propagation

3.2 Method

Given a graph $G = (V, E)$ and query entities Q, a personalization vector is defined so that only the query nodes receive restart probability mass. Personalized PageRank is then computed as

$$r = \alpha P^T r + (1 - \alpha)v,$$

where:

- P is the graph transition matrix,
- v is the query-based personalization vector,
- $\alpha = 0.85$ is the damping factor.

This means that the ranking process repeatedly returns to the query entities while also diffusing through nearby nodes. As a result, entities that lie on useful multi-hop pathways receive higher scores.

3.3 Multi-hop Query

The query used in this experiment is:

> What discoveries by Marie Curie led to later advances in medical imaging?

The query entities used as seeds are:

- `Marie Curie`,
- `medical imaging`.

3.4 Multi-hop Reasoning

Two main pathways are recovered by the graph:

Scientific pathway:

Marie Curie → radioactivity → radioisotopes → radiotracers → nuclear medicine → PET/SPECT → med

Direct application pathway:

Marie Curie → mobile X-ray units → battlefield radiography → medical imaging

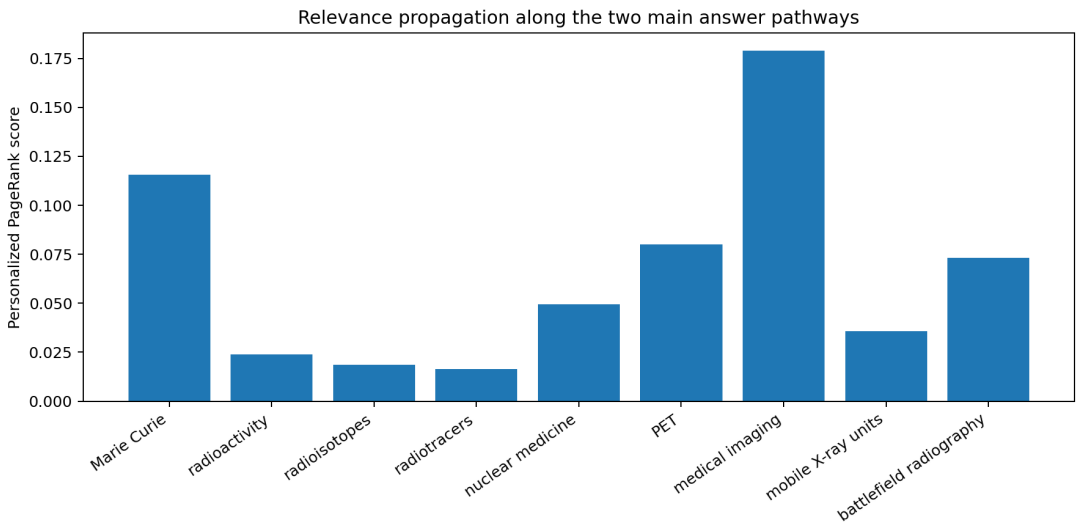

Figure 9: Personalized PageRank scores along the two main answer pathways

The plot above shows that relevance is not confined to the two seed entities only. Instead, it propagates into intermediate medical concepts, which is exactly what is needed for multi-hop retrieval.

3.5 Retrieved Answer

Marie Curie's discovery of radium and her pioneering work on radioactivity laid the scientific foundation for radiation-based medicine. These developments enabled the later use of radioisotopes and radiotracers in nuclear medicine, which led to imaging technologies such as PET and SPECT. In addition, her development of mobile X-ray units during World War I directly advanced the practical use of medical imaging.

3.6 Top-k Results

The top ranked nodes from the personalized PageRank run were:

- medical imaging,

- Marie Curie,
- functional imaging,
- radiotherapy,
- cancer diagnosis,
- PET,
- SPECT,
- battlefield radiography,
- nuclear medicine,
- radiation in medicine.

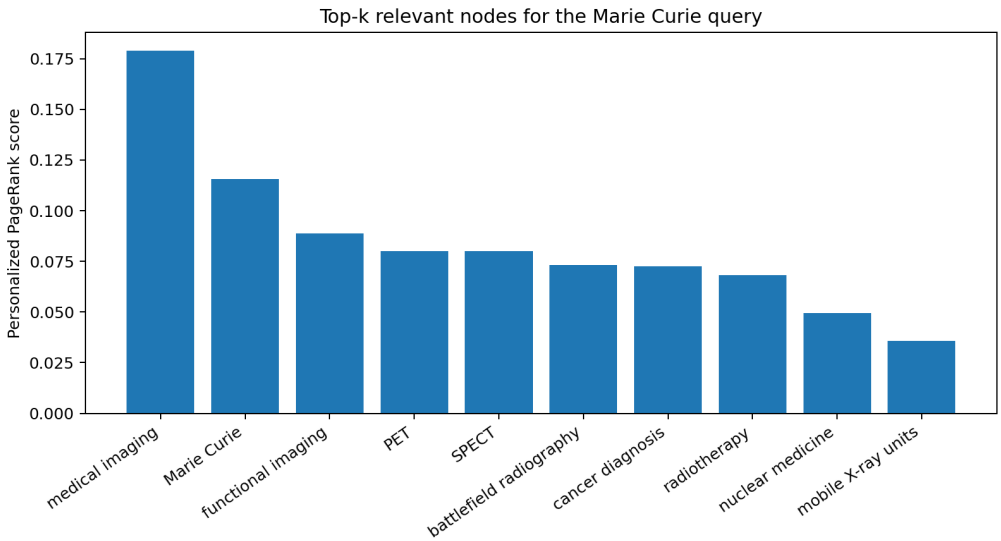

Figure 10: Top-k nodes ranked using personalized PageRank

3.7 Why Personalized PageRank Helps

To make the effect of personalization clearer, I compared query-focused personalized PageRank with ordinary global PageRank on the same graph.

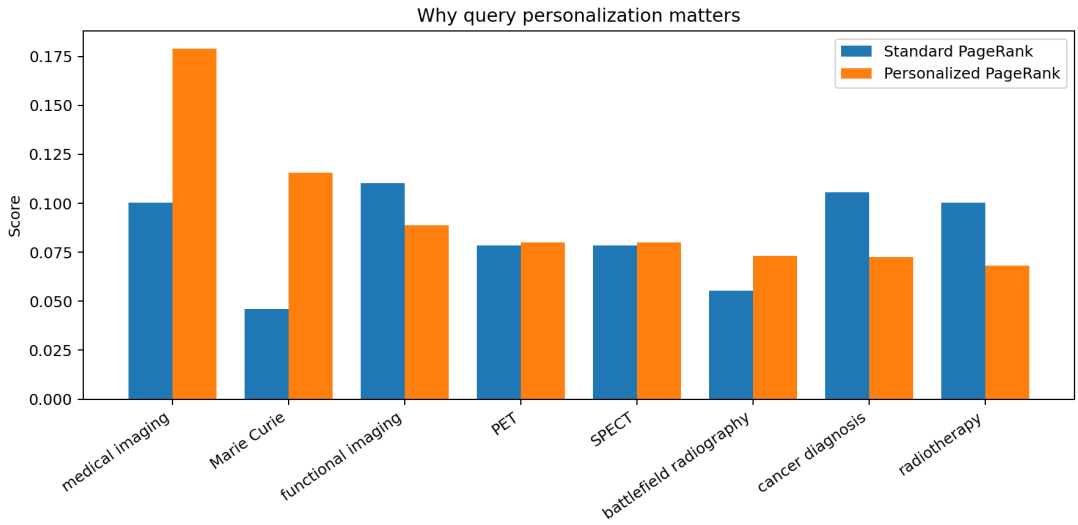

Figure 11: Comparison between standard PageRank and personalized PageRank on top query-relevant nodes

The comparison shows that personalized PageRank pushes the ranking toward nodes that are specifically useful for the Marie Curie + medical imaging query. This is important because a general graph-centrality score alone may favor globally important nodes, whereas personalized PageRank emphasizes nodes that are important *for this query context*.

3.8 Graph Visualization

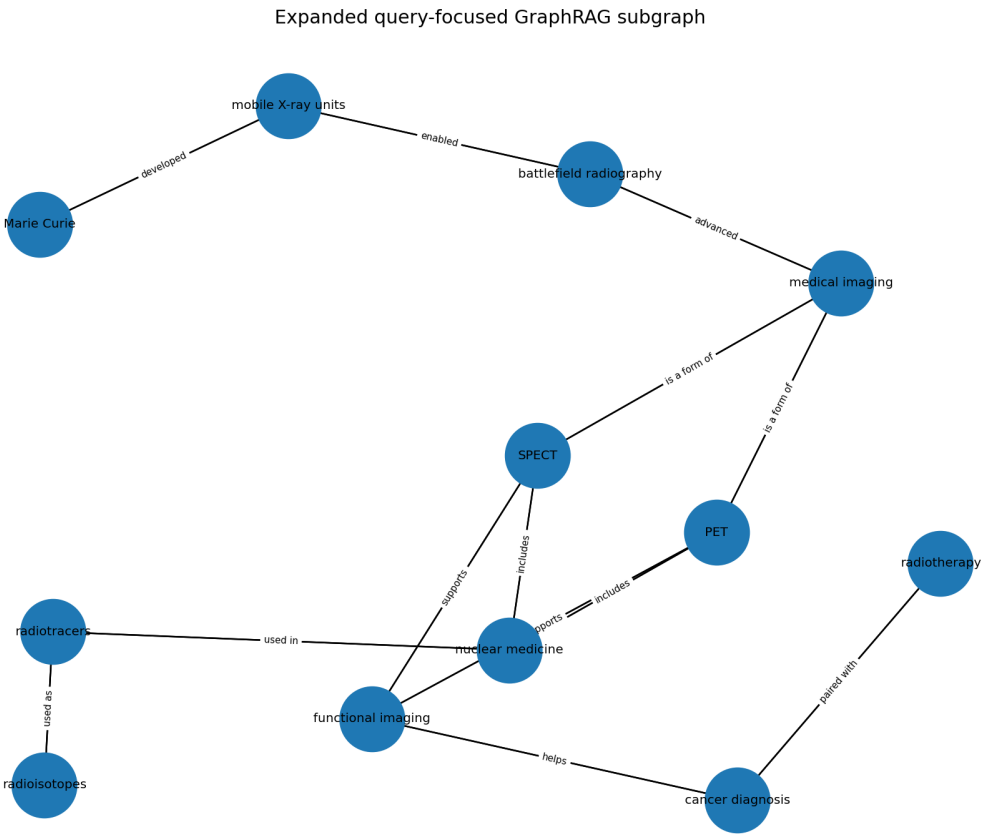

Figure 12: Expanded query-focused subgraph showing how relevance moves through multi-hop concepts

This graph view makes the retrieval process interpretable. The answer can be traced through explicit graph paths rather than treated as a black-box semantic match.

3.9 Discussion

This experiment highlights why PageRank is useful inside a GraphRAG pipeline:

- it supports **multi-hop retrieval** by spreading relevance through intermediate nodes,
- it improves **dependability** by ranking graph evidence rather than relying only on surface text similarity,
- it improves **interpretability** because the returned answer can be justified using explicit graph paths.

In other words, instead of only asking which nodes directly match the query, the method asks which nodes become important after relevance is allowed to flow through the graph from the query seeds.

3.10 Implementation

The accompanying program takes:

- a knowledge graph,
- a set of query entities,
- a value k,

and returns the top-k nodes ranked by personalized PageRank. It also generates report-ready visualizations automatically.

3.11 Conclusion

Personalized PageRank provides an effective mechanism for GraphRAG retrieval. By propagating importance from the query entities through graph connections, it can surface relevant intermediate concepts, recover multi-hop reasoning chains, and produce answers that are both accurate and explainable.

References

[1] circlemind-ai, *fast-graphrag*, GitHub repository. Available at: https://github.com/circlemind-ai/fast-graphrag

A Appendix

A.1 GitHub Repository

All Codes used in this report is available at my Github Repo:

https://github.com/zentorno365/Cloud-Computing-Assignment2.git

A.2 Appendix A: PageRank Implementation (Python)

```
#!/usr/bin/env python3
"""
PageRank with teleportation.

Supports:
1) power iteration on sparse graphs (large datasets)
2) exact closed-form solve on smaller graphs

Expected input format for Google 2002 datasets:
- first 4 lines are metadata/comments
- remaining lines: "src dst"

Usage examples
--------------
python pagerank_google.py web-Google_10k.txt --p 0.15 --iters 10 --topk 10 --
    exact
python pagerank_google.py web-Google.txt --p 0.15 --iters 10 --topk 10

Notes
-----
If teleport probability is p, then the damping factor is beta = 1 - p.
The PageRank vector x satisfies:

    x = (1-p) M^T x + (p/n) 1

where M is the row-stochastic hyperlink matrix after handling dangling nodes.

Closed form:
    x = (p/n) * (I - (1-p) M^T)^(-1) 1

This is practical only for small/medium graphs.
"""
from __future__ import annotations

import argparse
import math
from collections import defaultdict
from pathlib import Path
```

```python
from typing import Dict, List, Tuple

import numpy as np
from scipy import sparse
from scipy.sparse.linalg import spsolve


def read_google_graph(path: str | Path, skip_header: int = 4):
    """Read edge list, remap arbitrary node ids to 0..n-1, and return sparse
        adjacency."""
    node_to_idx: Dict[int, int] = {}
    edges_src: List[int] = []
    edges_dst: List[int] = []

    def get_idx(node_id: int) -> int:
        if node_id not in node_to_idx:
            node_to_idx[node_id] = len(node_to_idx)
        return node_to_idx[node_id]

    with open(path, "r", encoding="utf-8", errors="ignore") as f:
        for i, line in enumerate(f):
            if i < skip_header:
                continue
            line = line.strip()
            if not line or line.startswith("#"):
                continue
            a, b = line.split()
            u = get_idx(int(a))
            v = get_idx(int(b))
            edges_src.append(u)
            edges_dst.append(v)

    n = len(node_to_idx)
    data = np.ones(len(edges_src), dtype=np.float64)
    A = sparse.csr_matrix((data, (np.array(edges_src), np.array(edges_dst))),
        shape=(n, n))
    # collapse duplicate edges to 1 link count if duplicates exist
    A.data[:] = 1.0
    A.sum_duplicates()

    idx_to_node = np.empty(n, dtype=np.int64)
    for node_id, idx in node_to_idx.items():
        idx_to_node[idx] = node_id
    return A, idx_to_node
```

```python
def build_row_stochastic(A: sparse.csr_matrix) -> Tuple[sparse.csr_matrix, np.
    ndarray]:
    """Return row-stochastic M and dangling mask."""
    outdeg = np.asarray(A.sum(axis=1)).ravel()
    dangling = (outdeg == 0)
    safe_outdeg = outdeg.copy()
    safe_outdeg[dangling] = 1.0
    inv_outdeg = 1.0 / safe_outdeg
    M = sparse.diags(inv_outdeg) @ A
    return M.tocsr(), dangling


def pagerank_power(
    A: sparse.csr_matrix,
    p: float = 0.15,
    iters: int = 10,
    tol: float | None = None,
):
    """
    Power iteration for:
        x_{t+1} = (1-p) M^T x_t + (1-p) * dangling_mass/n * 1 + p/n * 1
    """
    n = A.shape[0]
    M, dangling = build_row_stochastic(A)
    x = np.full(n, 1.0 / n, dtype=np.float64)
    teleport = np.full(n, 1.0 / n, dtype=np.float64)

    history = []
    for t in range(iters):
        dangling_mass = x[dangling].sum()
        x_next = (1.0 - p) * (M.T @ x)
        x_next += ((1.0 - p) * dangling_mass + p) * teleport
        x_next /= x_next.sum()
        diff = np.abs(x_next - x).sum()
        history.append(diff)
        x = x_next
        if tol is not None and diff < tol:
            break
    return x, history


def pagerank_closed_form(A: sparse.csr_matrix, p: float = 0.15):
    """
    Exact solve for small graphs:
        x = (p/n) * (I - (1-p) P^T)^(-1) 1
    where P is the full stochastic matrix including dangling rows replaced by
        uniform rows.
```

```python
    """
    n = A.shape[0]
    M, dangling = build_row_stochastic(A)
    ones = np.ones(n, dtype=np.float64)

    # Build P^T = M^T + uniform contribution for dangling rows
    PT = M.T.tocsr()
    if dangling.any():
        dang_idx = np.where(dangling)[0]
        rows = np.repeat(np.arange(n), len(dang_idx))
        cols = np.tile(dang_idx, n)
        vals = np.full(n * len(dang_idx), 1.0 / n, dtype=np.float64)
        PT = PT + sparse.csr_matrix((vals, (rows, cols)), shape=(n, n))

    I = sparse.eye(n, format="csr", dtype=np.float64)
    rhs = (p / n) * ones
    x = spsolve(I - (1.0 - p) * PT, rhs)
    x = np.asarray(x, dtype=np.float64).ravel()
    x = np.maximum(x, 0)
    x /= x.sum()
    return x


def topk_report(x: np.ndarray, idx_to_node: np.ndarray, k: int = 10) -> List[
    Tuple[int, float]]:
    order = np.argsort(-x)[:k]
    return [(int(idx_to_node[i]), float(x[i])) for i in order]


def compare_vectors(a: np.ndarray, b: np.ndarray) -> dict:
    return {
        "l1": float(np.abs(a - b).sum()),
        "l2": float(np.linalg.norm(a - b)),
        "linf": float(np.max(np.abs(a - b))),
        "sum_a": float(a.sum()),
        "sum_b": float(b.sum()),
    }


def write_tsv(path: str | Path, x: np.ndarray, idx_to_node: np.ndarray):
    order = np.argsort(-x)
    with open(path, "w", encoding="utf-8") as f:
        for i in order:
            f.write(f"{x[i]:.12e}\t{int(idx_to_node[i])}\n")


def demo_small_graph():
```

```python
    """Built-in illustrative example."""
    # 0 -> 1,2,3 ; 1->0 ; 2->0 ; 3->0
    src = np.array([0, 0, 0, 1, 2, 3])
    dst = np.array([1, 2, 3, 0, 0, 0])
    A = sparse.csr_matrix((np.ones_like(src, dtype=float), (src, dst)), shape=(4,
        4))
    idx_to_node = np.arange(4)
    print("Small 4-node example")
    for p in [0.05, 0.15, 0.5, 0.9]:
        exact = pagerank_closed_form(A, p=p)
        approx, _ = pagerank_power(A, p=p, iters=200, tol=1e-14)
        cmp = compare_vectors(exact, approx)
        print(f"p={p:.2f} top={topk_report(exact, idx_to_node, 4)} l1={cmp['l1
            ']:.3e}")


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("path", nargs="?", help="path to web-Google_10k.txt or web
        -Google.txt")
    parser.add_argument("--p", type=float, default=0.15, help="teleport
        probability p")
    parser.add_argument("--iters", type=int, default=10, help="number of power
        iterations")
    parser.add_argument("--tol", type=float, default=None, help="optional stopping
         tolerance")
    parser.add_argument("--topk", type=int, default=10, help="number of top nodes
        to print")
    parser.add_argument("--exact", action="store_true", help="also compute exact
        closed form (small graphs only)")
    parser.add_argument("--out", type=str, default=None, help="optional TSV output
         path")
    parser.add_argument("--demo", action="store_true", help="run built-in
        illustrative example")
    args = parser.parse_args()

    if args.demo:
        demo_small_graph()
        return

    if not args.path:
        raise SystemExit("Please provide a graph file path, or use --demo.")

    A, idx_to_node = read_google_graph(args.path)
    n, m = A.shape[0], A.nnz
    print(f"Loaded graph: n={n:,} nodes, m={m:,} directed edges")
```

```python
    x_power, history = pagerank_power(A, p=args.p, iters=args.iters, tol=args.tol)
    print(f"Power iteration finished after {len(history)} steps")
    if history:
        print("Last L1 update:", history[-1])
    print(f"Top {args.topk} nodes by power iteration:")
    for node_id, score in topk_report(x_power, idx_to_node, args.topk):
        print(f"{node_id}\t{score:.12e}")

    if args.out:
        write_tsv(args.out, x_power, idx_to_node)
        print(f"Wrote ranked output to {args.out}")

    if args.exact:
        # Warn implicitly by running it only if requested.
        print("Computing exact closed-form solution...")
        x_exact = pagerank_closed_form(A, p=args.p)
        cmp = compare_vectors(x_exact, x_power)
        print("Comparison exact vs power:")
        for k, v in cmp.items():
            print(f"{k}: {v:.12e}")
        print(f"Top {args.topk} nodes by exact solve:")
        for node_id, score in topk_report(x_exact, idx_to_node, args.topk):
            print(f"{node_id}\t{score:.12e}")


if __name__ == "__main__":
    main()
```

A.3 Appendix B: GraphRAG Personalized PageRank Demo (Python)

```python
#!/usr/bin/env python3
"""
Enhanced GraphRAG-style Personalized PageRank demo.

Features:
- builds a small query-focused knowledge graph
- runs both standard and personalized PageRank
- returns top-k nodes for a multi-hop query
- generates multiple figures for a report:
    1. graphrag_pipeline_diagram.png
    2. graphrag_topk_scores_enhanced.png
    3. graphrag_personalized_vs_global.png
    4. graphrag_pathway_scores.png
    5. graphrag_query_subgraph_enhanced.png
"""
```

```python
from pathlib import Path
import matplotlib.pyplot as plt
import networkx as nx
import numpy as np


def build_demo_graph():
    G = nx.DiGraph()
    nodes = {
        "Marie Curie": "person",
        "Pierre Curie": "person",
        "polonium": "discovery",
        "radium": "discovery",
        "radioactivity": "concept",
        "radiation in medicine": "concept",
        "mobile X-ray units": "innovation",
        "battlefield radiography": "application",
        "radiotherapy": "application",
        "radioisotopes": "concept",
        "radiotracers": "concept",
        "nuclear medicine": "field",
        "PET": "imaging",
        "SPECT": "imaging",
        "medical imaging": "field",
        "cancer diagnosis": "application",
        "functional imaging": "application",
        "Curie Institute": "institution",
    }
    for n, kind in nodes.items():
        G.add_node(n, kind=kind)

    edges = [
        ("Marie Curie", "polonium", "discovered"),
        ("Marie Curie", "radium", "discovered"),
        ("Marie Curie", "radioactivity", "advanced science of"),
        ("Marie Curie", "radiation in medicine", "championed"),
        ("Marie Curie", "mobile X-ray units", "developed"),
        ("mobile X-ray units", "battlefield radiography", "enabled"),
        ("battlefield radiography", "medical imaging", "advanced"),
        ("radium", "radiotherapy", "enabled"),
        ("radioactivity", "radioisotopes", "underpins"),
        ("radioisotopes", "radiotracers", "used as"),
        ("radiotracers", "nuclear medicine", "used in"),
        ("nuclear medicine", "PET", "includes"),
        ("nuclear medicine", "SPECT", "includes"),
        ("PET", "medical imaging", "is a form of"),
        ("SPECT", "medical imaging", "is a form of"),
```

```
        ("PET", "functional imaging", "supports"),
        ("SPECT", "functional imaging", "supports"),
        ("functional imaging", "cancer diagnosis", "helps"),
        ("radiotherapy", "cancer diagnosis", "paired with"),
        ("radiation in medicine", "nuclear medicine", "influenced"),
        ("radiation in medicine", "radiotherapy", "influenced"),
        ("Curie Institute", "radiotherapy", "researches"),
        ("Curie Institute", "nuclear medicine", "researches"),
    ]
    for u, v, rel in edges:
        G.add_edge(u, v, relation=rel, weight=1.0)
        if not G.has_edge(v, u):
            G.add_edge(v, u, relation="reverse:" + rel, weight=0.35)
    return G


def run_rankings(graph, query_entities, alpha=0.85, k=10):
    personalization = {node: 0.0 for node in graph.nodes()}
    seed_weight = 1.0 / len(query_entities)
    for q in query_entities:
        personalization[q] = seed_weight

    ppr = nx.pagerank(graph, alpha=alpha, personalization=personalization, weight=
        "weight")
    std = nx.pagerank(graph, alpha=alpha, weight="weight")
    topk = sorted(ppr.items(), key=lambda x: x[1], reverse=True)[:k]
    return topk, ppr, std


def generate_figures(graph, topk, ppr, std, outdir="."):
    outdir = Path(outdir)
    outdir.mkdir(parents=True, exist_ok=True)

    # Pipeline diagram
    plt.figure(figsize=(10, 4))
    ax = plt.gca()
    ax.axis("off")
    boxes = [
        (0.05, 0.35, 0.18, 0.3, "Query\nMarie Curie +\nmedical imaging"),
        (0.29, 0.35, 0.18, 0.3, "Seed entities\nin knowledge graph"),
        (0.53, 0.35, 0.18, 0.3, "Personalized\nPageRank\npropagation"),
        (0.77, 0.35, 0.18, 0.3, "Top-k nodes\n+ answer paths"),
    ]
    for x, y, w, h, txt in boxes:
        rect = plt.Rectangle((x, y), w, h, fill=False, linewidth=2)
        ax.add_patch(rect)
        ax.text(x + w/2, y + h/2, txt, ha="center", va="center", fontsize=11)
```

```
for i in range(3):
    x1 = boxes[i][0] + boxes[i][2]
    x2 = boxes[i+1][0]
    ax.annotate("", xy=(x2-0.01, 0.5), xytext=(x1+0.01, 0.5),
                arrowprops=dict(arrowstyle="->", lw=2))
plt.title("GraphRAG retrieval pipeline with Personalized PageRank")
plt.tight_layout()
plt.savefig(outdir / "graphrag_pipeline_diagram.png", dpi=180)
plt.close()

# Top-k scores
plt.figure(figsize=(9, 5))
plt.bar([n for n, _ in topk], [s for _, s in topk])
plt.xticks(rotation=35, ha="right")
plt.ylabel("Personalized PageRank score")
plt.title("Top-k relevant nodes for the Marie Curie query")
plt.tight_layout()
plt.savefig(outdir / "graphrag_topk_scores_enhanced.png", dpi=180)
plt.close()

# Standard vs personalized PageRank
compare_nodes = [n for n, _ in topk[:8]]
x = np.arange(len(compare_nodes))
width = 0.38
plt.figure(figsize=(10, 5))
plt.bar(x - width/2, [std[n] for n in compare_nodes], width=width, label="
    Standard PageRank")
plt.bar(x + width/2, [ppr[n] for n in compare_nodes], width=width, label="
    Personalized PageRank")
plt.xticks(x, compare_nodes, rotation=35, ha="right")
plt.ylabel("Score")
plt.title("Why query personalization matters")
plt.legend()
plt.tight_layout()
plt.savefig(outdir / "graphrag_personalized_vs_global.png", dpi=180)
plt.close()

# Pathway scores
path1 = ["Marie Curie", "radioactivity", "radioisotopes", "radiotracers", "
    nuclear medicine", "PET", "medical imaging"]
path2 = ["Marie Curie", "mobile X-ray units", "battlefield radiography", "
    medical imaging"]
path_nodes = []
for n in path1 + path2:
    if n not in path_nodes:
        path_nodes.append(n)
plt.figure(figsize=(10, 5))
```

```
    plt.bar(path_nodes, [ppr[n] for n in path_nodes])
    plt.xticks(rotation=35, ha="right")
    plt.ylabel("Personalized PageRank score")
    plt.title("Relevance propagation along the two main answer pathways")
    plt.tight_layout()
    plt.savefig(outdir / "graphrag_pathway_scores.png", dpi=180)
    plt.close()

    # Expanded subgraph
    top_nodes = {n for n, _ in topk}
    top_nodes.update(["Marie Curie", "medical imaging", "radioisotopes", "
        radiotracers", "mobile X-ray units"])
    H = graph.subgraph(top_nodes).copy()
    pos = nx.spring_layout(H, seed=5, k=1.05)

    plt.figure(figsize=(10, 8))
    nx.draw_networkx_nodes(H, pos, node_size=1700)
    nx.draw_networkx_labels(H, pos, font_size=8)
    nx.draw_networkx_edges(H, pos, arrows=True, arrowstyle="-|>", arrowsize=14)
    edge_labels = {(u, v): d["relation"] for u, v, d in H.edges(data=True) if not
        d["relation"].startswith("reverse")}
    nx.draw_networkx_edge_labels(H, pos, edge_labels=edge_labels, font_size=6)
    plt.title("Expanded query-focused GraphRAG subgraph")
    plt.axis("off")
    plt.tight_layout()
    plt.savefig(outdir / "graphrag_query_subgraph_enhanced.png", dpi=180)
    plt.close()


if __name__ == "__main__":
    G = build_demo_graph()
    query_entities = ["Marie Curie", "medical imaging"]
    topk, ppr, std = run_rankings(G, query_entities, alpha=0.85, k=10)

    print("Top-k relevant nodes:")
    for node, score in topk:
        print(f"{node:25s} {score:.6f}")

    generate_figures(G, topk, ppr, std, ".")
```